SmartCart AI

Autonomous Predictive Waste & Executive Intelligence System for Supermarkets / Malls

Detailed Feature Handbook — Description, Tech Stack, Requirements & Process Flow

Project Type	Cross-Subject Capstone (Web Development, Artificial Intelligence, Data Warehousing & Mining)
Team Size	2-3 Members
Duration	8 Weeks
Scope	Single supermarket / mall unit (not multi-store)
Core Stack	React + TS + Tailwind + Recharts | Node/Express | Python FastAPI | PostgreSQL | Gemini API

How to read this handbook

Each feature that follows is documented as a self-contained unit: what it does and why it matters, the exact tools/libraries needed to build it, what data or prerequisites must exist before you can start, and a visual process flow with a step-by-step walkthrough. Read a feature fully before starting its build — each one names its dependencies on other features where relevant.

Feature Index

#	Feature
1	Daily AI Executive Briefing
2	Root Cause Tracer
3	Trained Risk-Scoring Model
4	Autonomy Trust Dial
5	Dynamic Auto-Discount Engine
6	Closed-Loop Reorder Engine
7	Live Decision Audit Trail
8	"Waste-Together" Association Mining
9	Store Risk Heatmap
10	OLAP Drill-Down Cube
11	Explainable Risk Breakdown
12	Computer Vision Freshness Scanner

1 Daily AI Executive Briefing

Dashboard - top panel

DESCRIPTION

This is the first thing a store manager sees each day. Instead of forcing them to read charts and interpret numbers, the system generates a short, prioritized, plain-English summary of what happened overnight and what deserves attention today — written the way a sharp junior executive would brief their manager. It typically lists 3-5 ranked points, each naming an issue, its likely cause, and the action already taken or recommended. This directly replaces the raw KPI-tile approach used in comparable systems with something that requires zero interpretation effort from the reader.

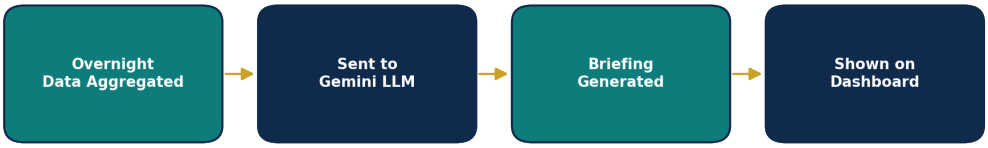
TECH STACK

- **Gemini API** — large language model that converts structured data into natural-language prose
- **Python (FastAPI)** — hosts the AI microservice that builds the prompt and calls Gemini
- **Node.js / Express** — aggregates the last 24 hours of DB activity into a compact JSON payload
- **PostgreSQL** — source of truth for risk scores, discount actions, and sales deltas
- **React** — renders the briefing as a styled card component on the dashboard
- **node-cron / scheduled job (optional)** — pre-generates the briefing once daily to avoid repeated LLM calls

REQUIREMENTS / PREREQUISITES

- Risk scores (Feature 3) and discount/audit logs (Features 5, 7) must already be populated — the briefing has nothing to summarize otherwise
- A Gemini API key configured in the FastAPI service environment
- A well-tested prompt template that enforces a strict, short output format (avoid the LLM rambling)
- A simple caching layer (even an in-memory dict keyed by date) so the briefing isn't regenerated on every page refresh

PROCESS FLOW



STEP-BY-STEP

Overnight Data Aggregated

A scheduled job or on-demand endpoint pulls the last 24 hours of risk-score changes, discount actions, and sales figures from PostgreSQL into one JSON object.

Sent to Gemini LLM

That JSON, plus a fixed prompt template instructing the model to output 3-5 ranked, plain-English points, is sent to the Gemini API from the FastAPI service.

Briefing Generated

Gemini returns a short structured text response, which is lightly parsed/cleaned and cached server-side.

Shown on Dashboard

The React dashboard fetches the cached briefing on load and renders it as the first thing the manager sees.

SUBJECT CONTRIBUTION: AI (natural language generation) · Web Dev (primary UI centerpiece)

2 Root Cause Tracer

Alerts Page - expandable item detail

DESCRIPTION

A risk score alone answers "is this bad?" but not "why does this keep happening?" This feature closes that gap: when a manager expands a Critical item, the system traces it back through supplier batch history, category-level patterns, and mined product associations to surface the most probable recurring cause — e.g. a specific supplier's recent batches consistently arriving with shorter shelf life. It turns a list of casualties into an actionable diagnosis.

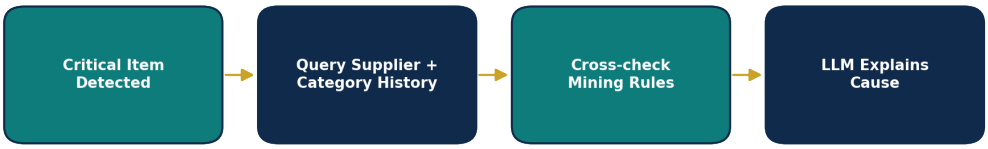
TECH STACK

- **PostgreSQL** — aggregation queries grouped by supplier_id, category, and store zone
- **Python** — rule-ranking logic that scores candidate causes by statistical strength
- **Output of Feature 8 (Association Mining)** — cross-checked against the root-cause candidates
- **Gemini API** — phrases the winning cause into one plain-English sentence

REQUIREMENTS / PREREQUISITES

- Association Mining (Feature 8) must be running first — this feature consumes its output
- At least a few weeks of historical supplier/category data to have meaningful patterns to trace
- Supplier and batch metadata must be captured at the point of stock receiving, not just at sale/waste time

PROCESS FLOW



STEP-BY-STEP

Critical Item Detected

The manager expands a product flagged Critical on the Alerts page, triggering a root-cause lookup request.

Query Supplier + Category History

PostgreSQL is queried for that product's recent supplier batches and category-wide waste statistics.

Cross-check Mining Rules

The result is compared against association rules already mined by Feature 8 to find the strongest matching pattern.

LLM Explains Cause

The top candidate cause is passed to Gemini, which phrases it as one clear, manager-readable sentence.

SUBJECT CONTRIBUTION: AI (reasoning & NLG) · DWM (data and rules feeding the reasoning)

3 Trained Risk-Scoring Model

Inventory Page - risk column on every product

DESCRIPTION

This is the analytical backbone of the whole system. Instead of the static weighted-sum formula used by comparable rule-based systems, every product is scored 0-100 for waste risk by a genuinely trained machine learning model. The model considers time-to-expiry, current stock versus projected demand, category-specific perishability, and historical waste rate, and its score updates continuously as new stock, sales, and expiry data flows in. This model's output feeds nearly every other feature in the system.

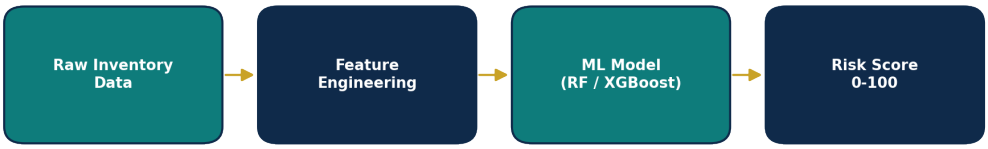
TECH STACK

- **Python** with **pandas** — data loading and feature engineering
- **scikit-learn** or **XGBoost** — the classification/regression model itself (Random Forest or Gradient Boosted Trees)
- **joblib** — persisting the trained model artifact for reuse without retraining on every request
- **FastAPI** — exposes a /predict-risk endpoint that the Node backend calls per product

REQUIREMENTS / PREREQUISITES

- A labeled or representative historical dataset (e.g. the 'Managing Perishable Inventory Data' dataset) with expiry, stock, and demand fields
- A defined feature-engineering pipeline (days-to-expiry, stock-to-demand ratio, category encoding, etc.)
- A train/test split and at least basic evaluation metrics (accuracy, F1, or RMSE depending on formulation) to justify the model choice in your report
- This is a prerequisite for Feature 11 (Explainable Risk Breakdown), which needs feature-importance output from this same model

PROCESS FLOW



STEP-BY-STEP

Raw Inventory Data

Product records (expiry date, current stock, category, recent sales) are pulled from PostgreSQL.

Feature Engineering

Raw fields are transformed into model-ready features: days-to-expiry, stock-to-demand ratio, category perishability index, historical waste rate.

ML Model (RF / XGBoost)

The trained model takes the feature vector and outputs a predicted risk value.

Risk Score 0-100

The raw model output is scaled to a 0-100 score and classified into Critical / High Risk / Warning tiers, then returned to the frontend.

SUBJECT CONTRIBUTION: AI (core model training deliverable)

4 Autonomy Trust Dial

Settings Page + persistent header control

DESCRIPTION

No manager will hand an AI system full control of pricing or purchasing on day one. This feature frames trust as something the system earns gradually: a five-level slider (L1 Advisory-Only → L5 Fully Autonomous) governs how much independent action the AI is permitted to take. At low levels it only suggests actions for manual approval; at higher levels it auto-executes low-risk decisions like small discounts while still asking approval for high-impact ones like donations or large write-offs. This is both a strong UX centerpiece and a genuine responsible-AI design pattern you can defend academically.

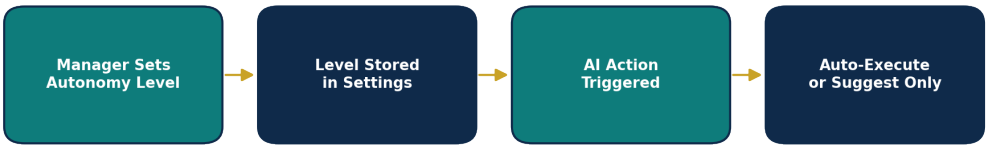
TECH STACK

- **React** — the slider component itself, with live state reflected instantly across the UI
- **PostgreSQL** — a Settings table storing the current autonomy level per store
- **Node.js / Express** — middleware/gating logic that checks the current level before allowing any AI-triggered action to execute automatically

REQUIREMENTS / PREREQUISITES

- Every other action-triggering feature (5, 6, and to a lesser extent 2) must be built to check this gate before executing — so this should be designed early, even if built visually later
- A clear internal definition of what counts as 'low-risk' vs 'high-impact' per action type

PROCESS FLOW



STEP-BY-STEP

Manager Sets Autonomy Level

The manager moves the slider on the Settings page (or header control) to their desired trust level.

Level Stored in Settings

The new level is saved to the Settings table in PostgreSQL immediately.

AI Action Triggered

Elsewhere in the system, an event occurs that could warrant automatic action (e.g. a product turns Critical).

Auto-Execute or Suggest Only

The action-handling code reads the current autonomy level and either executes the action automatically or queues it as a suggestion awaiting manual approval.

SUBJECT CONTRIBUTION: AI (responsible-AI / human-in-the-loop logic) · Web Dev (signature UX interaction)

5 Dynamic Auto-Discount Engine (Self-Learning)

Alerts Page (action button) + Analytics Page (performance chart)

DESCRIPTION

When a product crosses a risk threshold, this engine doesn't just flag it — it calculates and, depending on the Trust Dial's level, applies a specific discount percentage automatically. Crucially, the discount percentage itself is learned rather than fixed: the engine tries different discount levels per category over time, tracks which ones actually cleared stock before expiry, and shifts future recommendations toward what has worked. This directly answers the reference paper's own stated limitation that its recommendation engine is 'rule-driven' rather than adaptive.

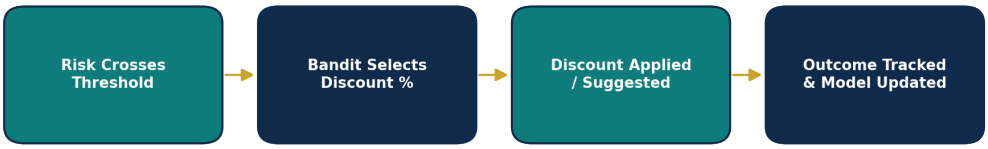
TECH STACK

- **Python** — implements a simple multi-armed bandit (epsilon-greedy is sufficient) to select discount levels
- **PostgreSQL** — logs every discount decision and its eventual outcome (did the stock clear before expiry?)
- **FastAPI** — exposes the discount-decision endpoint
- **React** — the inline 'Discount' action button and the discount-performance chart on Analytics

REQUIREMENTS / PREREQUISITES

- The Trained Risk-Scoring Model (Feature 3) must be producing risk tiers already, since this engine only triggers above a risk threshold
- A way to track sell-through outcome per discount event (even a simple 'sold before expiry: yes/no' flag is enough to start learning from)
- Category-level grouping so the bandit can learn separately for, say, Bakery vs Dairy rather than treating all products identically

PROCESS FLOW



STEP-BY-STEP

Risk Crosses Threshold

A product's risk score (from Feature 3) enters the Critical or High Risk tier, triggering the discount-decision logic.

Bandit Selects Discount %

The epsilon-greedy algorithm picks a discount percentage for that product's category, mostly favoring levels that have historically worked but occasionally exploring new ones.

Discount Applied / Suggested

Depending on the Autonomy Trust Dial (Feature 4), the discount is either applied automatically or queued as a suggestion.

Outcome Tracked & Model Updated

Once the item is sold or expires, the outcome is logged, and the bandit's internal statistics for that category/discount-level pair are updated for future decisions.

> **SUBJECT CONTRIBUTION:** AI (adaptive, probabilistic decision-making — a direct improvement over the reference paper's static rule-based approach)

6 Closed-Loop Reorder Engine

Purchases / Reorder Page - suggested quantities panel

DESCRIPTION

Discounting at-risk stock is defense; this feature is offense. Rather than only reacting to products that are already at risk, the system uses what it has learned from recent waste patterns to adjust recommended purchase-order quantities for the next buying cycle — closing the loop between what went wrong this week and what gets ordered next week, so the same over-ordering mistake doesn't repeat.

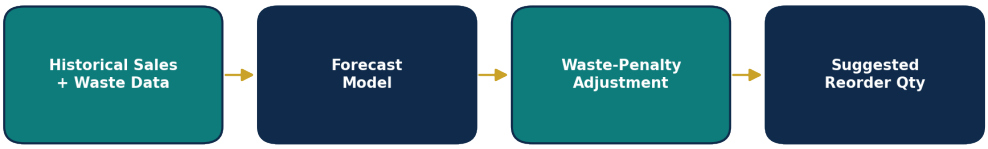
TECH STACK

- **Python** — a lightweight forecasting model (moving average or exponential smoothing is sufficient; Prophet if you want more rigor)
- **PostgreSQL** — historical sales and waste tables as model input
- **FastAPI** — serves the suggested-order-quantity endpoint
- **React** — an editable suggested-order table the manager can review and adjust before confirming

REQUIREMENTS / PREREQUISITES

- At least several weeks of historical sales AND waste data per product/category — forecasting quality depends heavily on data volume here
- A defined 'waste-penalty factor' formula: how much should recent over-ordering reduce the next suggested quantity

PROCESS FLOW



STEP-BY-STEP

Historical Sales + Waste Data

Recent sales volume and waste volume per product/category are pulled from PostgreSQL.

Forecast Model

A trend/seasonality-aware model projects expected demand for the next ordering cycle.

Waste-Penalty Adjustment

The raw forecast is adjusted downward for products/categories that have recently shown high waste rates, to avoid repeating the over-order.

Suggested Reorder Qty

The adjusted quantity is surfaced in an editable table for the manager to approve or override.

SUBJECT CONTRIBUTION: AI (forecasting model) · DWM (historical data aggregation)

7 Live Decision Audit Trail

Dashboard side panel + dedicated Audit Log page

DESCRIPTION

Autonomous or semi-autonomous AI actions only earn a manager's trust if every decision is fully visible and explainable after the fact. This feature is a scrolling, timestamped feed logging every action the system has taken — discount applied, order adjusted, donation triggered — along with the reasoning behind it. It is what makes the Autonomy Trust Dial (Feature 4) safe to turn up.

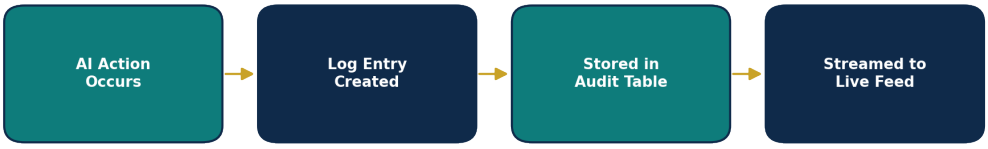
TECH STACK

- **PostgreSQL** — an audit/notification table storing action type, target product, reasoning text, and the autonomy level at the time
- **Node.js / Express** — every action-triggering endpoint writes a structured log entry as a side effect
- **React** — a live feed component, either polling periodically or using a simple event-based update pattern, plus toast notifications for new critical entries

REQUIREMENTS / PREREQUISITES

- Every feature that takes an autonomous action (5, 6, and parts of 2) must be built to write a log entry — treat this as a shared utility function used across the backend, not a standalone feature built in isolation

PROCESS FLOW



STEP-BY-STEP

AI Action Occurs

Any autonomous or suggested action elsewhere in the system (e.g. Feature 5 applying a discount) completes.

Log Entry Created

The triggering function calls a shared logging utility with the action type, target product, and a short reasoning string.

Stored in Audit Table

The entry is written to PostgreSQL with a timestamp.

Streamed to Live Feed

The React frontend fetches new entries (via polling or on next load) and renders them at the top of the scrolling feed, with a toast for high-priority entries.

SUBJECT CONTRIBUTION: Web Dev (transparency & trust-building UI)

8 "Waste-Together" Association Mining

Insights Page - association rules panel

DESCRIPTION

This feature mines historical waste-event data to surface non-obvious patterns — for example, products from the same supplier or shelf zone that tend to spoil together, even if they belong to different categories. These mined rules are what power the Root Cause Tracer (Feature 2), turning raw co-occurrence statistics into a genuine diagnostic tool, and they are also this project's core, dedicated Data Warehousing & Mining deliverable.

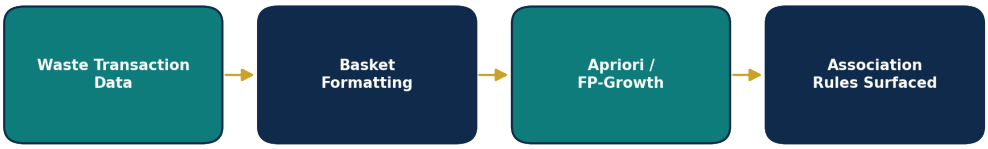
TECH STACK

- **Python** with **mlxtend** — provides ready-made Apriori and FP-Growth implementations plus association-rule generation (support, confidence, lift)
- **pandas** — reshaping raw waste events into the 'basket' (transaction) format the algorithms require
- **PostgreSQL** — stores the resulting rules for reuse instead of recomputing on every request

REQUIREMENTS / PREREQUISITES

- Waste events need to be groupable into meaningful 'transactions' — e.g. all products that went to waste on the same day, from the same store zone, or the same supplier delivery
- A dataset with enough transactions to produce statistically meaningful rules (the purpose-built 11-item Grocery Store dataset is ideal for a clean classroom-scale Apriori demo)

PROCESS FLOW



STEP-BY-STEP

Waste Transaction Data

Historical waste events are pulled from PostgreSQL, grouped by a meaningful transaction unit (e.g. per day per store zone).

Basket Formatting

The grouped data is reshaped into the one-hot-encoded 'basket' format Apriori/FP-Growth expects.

Apriori / FP-Growth

The mining algorithm identifies frequent itemsets and generates association rules ranked by support, confidence, and lift.

Association Rules Surfaced

The strongest rules are stored and displayed as readable statements on the Insights page, and made available to Feature 2's root-cause logic.

> **SUBJECT CONTRIBUTION:** DWM (core association rule mining requirement)

9 Store Risk Heatmap

Analytics Page - interactive store layout view

DESCRIPTION

Numbers and lists don't convey where risk physically lives in the store. This feature groups products and store zones by similarity in waste behavior using clustering, then maps that risk visually onto a simplified store layout with color intensity per zone — letting a manager see at a glance which physical areas need the most attention, rather than scanning a table.

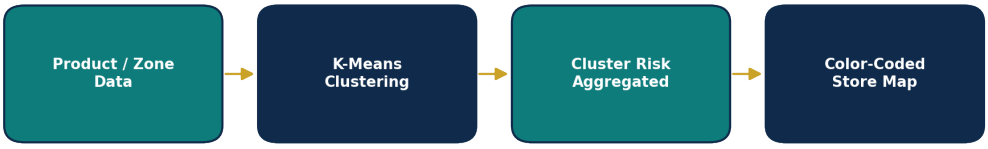
TECH STACK

- **Python** with **scikit-learn** — K-Means clustering grouping products/zones by waste-behavior similarity
- **PostgreSQL** — zone metadata joined against clustering output
- **React + SVG** — renders a simplified store-layout graphic with dynamic fill colors per zone based on aggregated cluster risk

REQUIREMENTS / PREREQUISITES

- Products must be mapped to physical store zones/aisles in your schema — if this mapping doesn't exist yet, it needs to be added before this feature can work
- Enough data points per zone for clustering to be meaningful (very sparse zones will cluster poorly)

PROCESS FLOW



STEP-BY-STEP

Product / Zone Data
Product risk scores and their assigned store zones are pulled from PostgreSQL.

K-Means Clustering
Products and/or zones are clustered by waste-behavior similarity using scikit-learn's K-Means.

Cluster Risk Aggregated
Average risk level is computed per cluster/zone.

Color-Coded Store Map
The aggregated risk per zone is mapped to a color intensity and rendered on the SVG store layout in the React frontend.

> **SUBJECT CONTRIBUTION:** DWM (clustering) · Web Dev (spatial visualization UX)

10 OLAP Drill-Down Cube

Analytics Page - main chart area

DESCRIPTION

Static, pre-aggregated charts only answer the question you thought to ask in advance. This feature gives managers a rollup analytics view across Category, Store Zone, and Time that they can click into — starting from a high-level category view and drilling down into a specific store zone, then a specific day — the classic OLAP cube pattern, powered by your star-schema data warehouse rather than the raw OLTP tables.

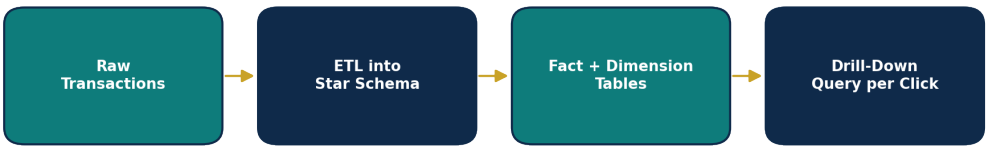
TECH STACK

- **PostgreSQL** — a star schema with a central fact table (fact_waste_events) and dimension tables (dim_product, dim_store, dim_time, dim_category)
- **SQL GROUP BY / ROLLUP** — parameterized aggregation queries per drill level
- **Node.js / Express** — exposes drill-level-aware query endpoints
- **Recharts** — the interactive chart component, updating on click-to-drill interactions

REQUIREMENTS / PREREQUISITES

- A working ETL process (even a simple scheduled script) that populates the star schema from the OLTP tables — this needs to exist before any drill-down query can run
- This is the single biggest prerequisite dependency in the whole project: several other features (2, 6, 9) benefit from or depend on this warehouse layer existing early

PROCESS FLOW



STEP-BY-STEP

Raw Transactions

OLTP data (individual sales, stock updates, waste events) is captured as it happens.

ETL into Star Schema

A scheduled or on-demand process transforms and loads this raw data into the fact and dimension tables.

Fact + Dimension Tables

The warehouse now holds pre-structured, aggregation-ready data keyed by product, store, time, and category.

Drill-Down Query per Click

When a manager clicks into a category or zone in the UI, a parameterized query at the appropriate granularity is run against the warehouse and rendered as an updated chart.

SUBJECT CONTRIBUTION: DWM (core data warehouse requirement) · Web Dev (interactive chart UX)

11 Explainable Risk Breakdown

Inventory Page - product detail modal

DESCRIPTION

A bare risk number invites distrust — 'why is this 87 and not 60?' This feature answers that directly: every product's score is broken down into a visual waterfall of contributing factors (expiry proximity, demand mismatch, category perishability, historical waste rate), followed by a one-line plain-English explanation. It turns the trained model (Feature 3) from a black box into something a manager can actually reason about and trust.

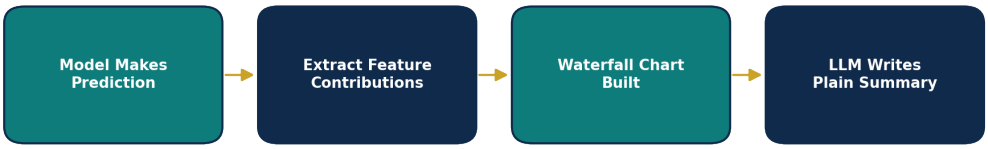
TECH STACK

- **SHAP** (or the model's built-in feature_importances_ as a simpler fallback) — computes per-prediction factor contributions
- **Python / FastAPI** — computes and serves the breakdown alongside the risk score
- **Recharts** — renders the waterfall/bar visualization
- **Gemini API** — converts the top contributing factors into a single explanatory sentence

REQUIREMENTS / PREREQUISITES

- Directly depends on Feature 3 (Trained Risk-Scoring Model) — the model must already be trained and serving predictions before this can be added
- If using SHAP, budget extra time: SHAP computation can be slow for tree ensembles on large batches, so consider computing it only on-demand for a single expanded product rather than for the entire inventory list

PROCESS FLOW



STEP-BY-STEP

Model Makes Prediction

The trained risk model (Feature 3) produces a risk score for a specific product, as usual.

Extract Feature Contributions

SHAP (or feature importances) is computed for that specific prediction, showing how much each input factor pushed the score up or down.

Waterfall Chart Built

The contributions are rendered as a waterfall chart showing the cumulative build-up to the final score.

LLM Writes Plain Summary

The top 1-2 contributing factors are passed to Gemini to generate one short, plain-English explanatory sentence displayed above the chart.

SUBJECT CONTRIBUTION: AI (explainable AI) · Web Dev (trust-building detail UI)

12 Computer Vision Freshness Scanner

Inventory Page - 'Scan Item' action / dedicated modal

DESCRIPTION

Expiry dates alone don't always reflect actual product condition. This feature lets a manager photograph or upload an image of a perishable item, and a trained image-classification model estimates its freshness/spoilage stage directly from visual cues (browning, wilting, discoloration), feeding this as an additional signal into the product's risk score. Notably, the reference paper's literature review cites this exact technique as an emerging capability but never implements it — building it here is a genuine, defensible extension beyond the paper's own scope.

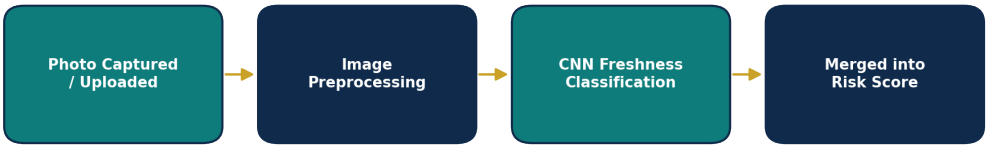
TECH STACK

- **TensorFlow/Keras** or **PyTorch** — deep learning framework for the image classifier
- **MobileNetV2 (transfer learning)** — a lightweight pretrained CNN fine-tuned on a fresh/rotten produce dataset, avoiding the need to train from scratch
- **FastAPI** — exposes a /scan-freshness endpoint accepting an uploaded image
- **React** — a camera-capture or file-upload widget calling the endpoint and displaying the result

REQUIREMENTS / PREREQUISITES

- A labeled fresh/rotten produce image dataset (several public datasets exist for this exact task) for fine-tuning
- This is a second, independent trained model separate from Feature 3 — budget real time for data preprocessing, training, and validation; this is the most ML-engineering-heavy feature in the project
- A defined rule for how the freshness output merges into the existing tabular risk score (e.g. as a score adjustment/multiplier rather than a full replacement)

PROCESS FLOW



STEP-BY-STEP

Photo Captured / Uploaded

A manager captures or uploads a photo of a perishable item through the React widget.

Image Preprocessing

The image is resized/normalized to match the model's expected input format.

CNN Freshness Classification

The fine-tuned MobileNetV2 model classifies the image into a freshness category (e.g. Fresh / Aging / Spoiled).

Merged into Risk Score

The classification result is combined with the existing tabular risk score (Feature 3) as an adjustment factor, and the updated score is saved and displayed.

> **SUBJECT CONTRIBUTION:** AI (second, independent trained model — a genuine addition beyond the reference paper's cited-but-unbuilt technique)